

AI 시대 SW Test, 산출물 Verify, Report, PR 전략

Agenda

- I. AE 관점에서 Unit Test
- II. AI가 수행한 테스트도 사람의 검토가 필요할까?
- III. 기술부채와 인지적 부채
- IV. Verify Harness 도입
- V. AI 작업의 보고 체계 변경
- VI. AI 시대에 맞는 PR 전략 - 감리

교육 내용 및 학습 목표

- Unit Test와 E2E 테스트를 Agentic Engineering 관점에서 살펴본다.
- 이후 기술부채와 인지적 부채 개념을 살펴보고,
- AI 산출물에 대한 검증, Report 체계, 그리고 AI 시대에 맞는 Pull Request를 살펴본다.

Chapter 01

AE 관점에서 Unit Test

SW 품질을 위한 Agentic Engineering

Unit Test의 목적

Unit Test의 목적은 코드의 품질을 지속적으로 보장하는 데 있다.
핵심 목적은 다음 두 가지로 정리할 수 있다.

Regression (회귀 검증)

- 기존에 정상 동작하던 기능이 변경 이후에도 동일하게 동작하는지 확인
- 리팩토링이나 기능 추가 이후, 의도치 않은 버그를 방지하는 역할

Correctness (정확성 검증)

- 현재 작성된 코드가 요구사항에 맞게 정확하게 동작하는지 검증
- 다양한 입력과 조건에 대해 기대한 결과를 올바르게 반환하는지 확인

Regression의 정의

기존 기능이 망가지지 않았는지 확인하는 테스트

Regression 비유

- 부품에 보호 코팅을 바르는 것으로 비유할 수 있다.
 - 코팅이 손상되면 즉시 비상벨이 울린다.
 - 코팅을 바르는 것 = Regression 테스트를 구축하는 것
 - 이후 다른 부품을 수정하더라도, 코팅이 손상되는지(기존 기능이 깨지는지) 지속적으로 확인할 수 있다.
- Regression Test는 코드가 조금 변경될 때마다 자주 돌려주어야 한다.

Unit Test는 Correctness 효과가 적다.

이미 예상하고 만든 내용을 검증하는 경우가 많다.

- 예상 범주 내에서 Test Case를 만들기에 항상 Pass한다.

테스트 범위가 좁다.

- Unit Test는 하나의 Unit만 검증한다.
- 실제 환경에서 버그는, 여러 유닛들이 복잡한 상호작용을 하는 과정에서 발생

코드 작성자의 편향이 반영됨

- 코드 작성시, "이 정도면 버그가 나지 않고, 잘 동작하겠네?"라는 편향된 생각
- 그 이후로 방어가 가능할 것이라고 예상이 되는 수준에서 테스트를 만든다.

(코드 개발자가 아닌, QA가 Test Case를 만든다면 더 공격적인 테스트를 만들었을 것)

제작한 사람이 아닌, AI의 Test 필요

개발자가 아닌 관점의 테스트가 필요
효과적인 정확도(Correctness) 테스트

AI로 Correctness를 넘어서 안전성 테스트 (Safety Testing)를 진행
망가뜨리려는 시도를 통해 시스템의 안정성을 검증

- 이상한 값을 마구 넣어보기
- 예외 일으켜보기, 잘못된 입력 넣어보기
- 극단적인, 못된 사용자가 있다고 가정하고 테스트를 해보기

AI는 사람의 편향을 보완하여,
더 넓고 공격적인 테스트를 생성 및 수행하는 데 유용

공격적인 Test는, 과연 Good Test일까? - No 일 가능성이 큼

불량을 찾기 위해 공격적인 Test Case를 만드는 접근은 충분히 의미가 있다.

But 리팩토링 내성이 낮은 Test의 가능성이 높음

- 특정 구현에 강하게 의존할 가능성 높음
- 처리 방식 등 내부 로직이 조금이라도 바뀌면 쉽게 깨지는 테스트일 가능성이 높음

이 테스트는 임시 테스트로 분류하자!

- Git에 포함되지 않는 임시 경로(/tmp 등)에서 테스트 코드 만들어 실행

AE 관점 - 생성된 Test 코드를 어디까지 이해해야 할까?

AI가 만든 Correctness 목적의 Test 코드도 리뷰는 필요
But 가볍게만 이해한다.

- 무엇을 검증하는지, 의도를 중심으로 이해하면 충분
- 소스코드를 디테일하게 이해할 필요는 없음

Test Fail 발생 시 대응 방법

- 재현 가능한 최소 테스트를 생성
- 어떤 조건에서 Fail되는지 AI와 소통을 통해 정확히 원인을 파악
- 이후 대책에 대해 AI와 소통하고, 협의가 되면 Plan 문서를 작성
- AI Action 수행을 하고, 수정된 코드에 대해 리뷰하고 재현 테스트에 성공하는지 확인

[도전] Regression / Safety Test 수행

기존에 제작한 Json + Console CRUD App 에
Regression / Safety Test 수행해보기

Chapter 02

AI가 수행한 테스트도 사람의 검토가 필요할까?

SW 품질을 위한 Agentic Engineering

AI가 테스트를 잘 하더라도, 사람이 수동적으로 검토는 필요하다.

- AI가 어떤 테스트를 했는지, 사람의 확인이 필요
- AI 없이 사람이 직접 테스트도 해보는 것도 권장

AI가 스스로 테스트 해보았다면, 사람의 동작 테스트가 필요할까?

AI가 스스로 E2E 테스트를 수행하고 (verify) 답변하면, 인간의 리뷰 자체

- (인간이 직접 크롬 띄워서 동작테스트 해보는 행동)가 필요 없는 거 아닌가?

사람이 직접 동작을 확인하는 과정은 단순한 테스트가 아니라, 전체 흐름을 보며 개선점과 다음 방향을 고민하는 검토 활동이다.

사이먼 윌리슨은 E2E 테스트가 자동화되더라도, 최종적으로는 사람이 눈으로 직접 확인하는 과정을 선호한다.

사람이 테스트를 직접 해보는 행동은, 사람이 주도권을 가지고 직접 검토를 하는 행위이며, 이는 Agentic Engineering 관점에서도 중요한 행동이라고 할 수 있다.

사이먼 윌리슨의 Showboat 예시

사람이 직접 테스트 해보지 않고도, AI가 테스트한 이력들을 다 남겨서, 스크린샷하고, 이걸로 리뷰도 가능하다. 이 행동도 Agentic Engineering 이고, 이걸로 사람이 직접 동작테스트를 대신할 수도 있다.

Chapter 03

기술부채와 인지적 부채

SW 품질을 위한 Agentic Engineering

기술부채(Technical Debt)란?

지금 빨리 만들기 위해 하는 행동들이, 나중에 시간을 더 많이 들이게 되는 상황을 “기술 부채가 발생했다”고 한다.

급하게 개발하면서 구조를 대충 만들고, 테스트를 미리 하지 않고,
임시 코드를 넣어서 일단 돌아가게는 한 상태일 때를 예로 들 수 있다.

이는 차후에 수정하거나 유지보수 할 때 더 큰 SW 비용을 가져온다.

부채라고 부르는 이유

당장의 시간이 부족으로 지저분한 코드들을 양산
이후 유지보수에 많은 시간을 소비하며 후회

이는 은행에서 돈을 빌리는 상황과 비슷하다.
빌릴 때는 쉽게 결정하지만, 갚을 때 그에 대한 대가를 치뤄야 한다.

은행에서 돈을 빌리는 행동은 심적 부담감이 큰 것처럼,
SW에서도 지저분하게 코딩하는 것을 가볍게 여기지 말라는 의도로 만들어진 용어

SW 공학에서 의미

기술 부채는 원래 SW공학에서 사용되는 용어로 다음과 같은 상태를 뜻한다.

- 이해하기 어려운 소스코드 구조
- 확장과 수정이 힘든 설계
- 복잡도가 높은 코드
- 빠르게 만든 임시 로직 및 누적된 버그

이러한 요소들이 쌓이면서 전체 시스템 유지보수 비용이 증가되는 상태

해결방법 : 리팩토링

구조를 정리하고, 복잡도를 줄이고, 테스트를 보완하는 과정으로 부채를 줄여 나갈 수 있다.

인지적 부채 (Cognitive debt)

인지적 부채는 Coding Agent로 인해 빠른 속도로 개발되는 과정에서, 코드를 충분히 이해하지 못한 상태로 누적되는 부담을 의미

1. 개발자 본인의 소스코드 이해가 낮아지는 현상
2. 내 코드에 대한 이해 수준 차이가 커지는 현상

이러한 인지적 부채는 다음과 같은 문제가 발생한다.

1. 팀 내 커뮤니케이션 혼란
2. 유지보수시 높은 부담, 얼마나 걸릴지, 버그 위험도가 예측이 안됨
3. 시스템 전반 품질 저하

인지적 부채는 기술 부채의 한 종류로 분류되지 않는다.

인지적 부채는 기술 부채와 비슷하지만, 두 가지를 구분하고 있다.

기술부채는 리팩토링을 통해 해결하지만,

인지적 부채는 학습, 리뷰, 문서화 등을 통해 해결해야 한다.

기술부채

소스코드 자체 문제에 포커스가 맞춰져있다.

급하게 만든 코드

이해하기 어려운 코드

중복 및 복잡도가 높은 코드

인지적 부채

사람이 코드를 이해하지 않은 문제에
포커스가 맞춰져 있다.

담당자가 코드 설명 못함

수정 시 어떤 리스크가 있을지 모름

팀원이 어떤 개발하는지 모름

인지적 부채를 해결하기 위한 방법 - 리팩토링 / Agentic Engineering

Coding Agent 사용에서 가장 큰 문제 중 하나는 인지적 부채

마틴파울러는 기술적 부채 뿐만 아니라, 인지적 부채를 해결하기 위해서도
페어 프로그래밍 / 리팩토링 / TDD 같은 실천 중심의 개발 방식을 강조

Agentic Engineering은 “**이해하지 못한 코드를 그대로 사용하지 않는다.**”는 원칙을 기반으로,
AI가 생성한 코드를 반드시 리뷰하고 최종 승인하는 과정을 포함

개인 수준에서 발생하는 인지적 부채를 줄이는 데 효과적인 접근 방식

Agentic Engineering 만으로는 팀내 인지적 부채를 해결하기 어렵다.

Agentic Engineering 만으로 팀 전체의 인지적 부채를 해결하기에는 한계가 있다.

팀 차원에서는 인지적 부채를 해결하기 위해 여전히 다음과 같은 노력이 필요하다.

- 공통된 이해의 형성
- 지식 공유
- 문서화 및 커뮤니케이션
- 이해 없이 속도만 높이는 개발팀은 지속 가능하지 않다.

인지적 부채를 줄이기 위해서는 팀 단위의 전략 예시

1. 변경 내용뿐 아니라 변경 이유까지 함께 문서화한다.
(AI에게 변경 이유를 상세히 작성하도록 지시하는 것도 좋은 방법이다)
2. 변경 이력과 의사 결정 근거를 체계적으로 기록한다. (History 관리)
3. 코드 리뷰 및 회고, 지식 공유 세션을 통해 팀 내 공통된 이해를 지속적으로 재구축해야 한다.
4. 정기적인 코드 점검 및 시각화 문서 만들기

팀의 인지적 부채 위험 신호

한 사람에게만 의존하는 암묵적 지식이 늘어나기 시작하면,
시스템이 점점 블랙박스처럼 느껴지게 된다.

이는 팀 내에 공유된 이해가 약화되고 있다는 신호이며,
인지적 부채가 빠르게 쌓이고 있다는 경고이다.

Chapter 04

Verify Harness 도입

SW 품질을 위한 Agentic Engineering

개발 —> Verify —> 코드 리뷰 프로세스

AI가 만든 코드를 인간이 리뷰하기 전, AI 스스로 충분히 테스트가 되어 있어야 한다.
PLAN.md를 작성하고 Action을 수행할 차례에서 검증을 포함하여 지시한다.

기존 : Phase1 개발해

변경 : Phase1 개발하고 Verify까지 수행해 (가능하다면 Sub-Agent로 동작)

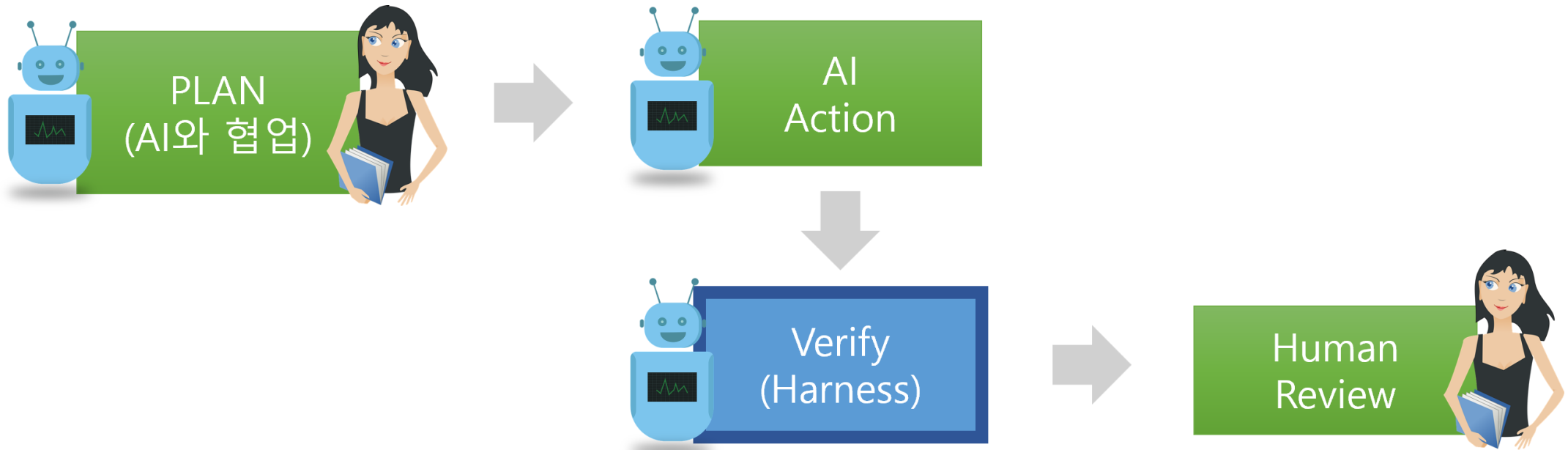
- Verify는 기본 동작 검증(Correctness)이 될 수 있다.
- 필요 시, 사람이 확인해야 할 내용을 AI가 선행 검증하도록 할 수 있다.
- 필요 시, /tmp 에서 공격적인 Safety Test를 수행하도록 지시할 수 도 있다.

Test가 정상적으로 수행된 이후, 코드 리뷰를 진행한다.

개발 —> Verify —> 코드 리뷰 프로세스

Verify 행동은 인간 코드 리뷰 이전에 진행되어야 하며, 그림으로 나타내면 아래와 같다.
Verify 단계는 AI 결과물이 안전하게 동작하도록 하는 구조이며 Harness 라고 볼 수 있다.

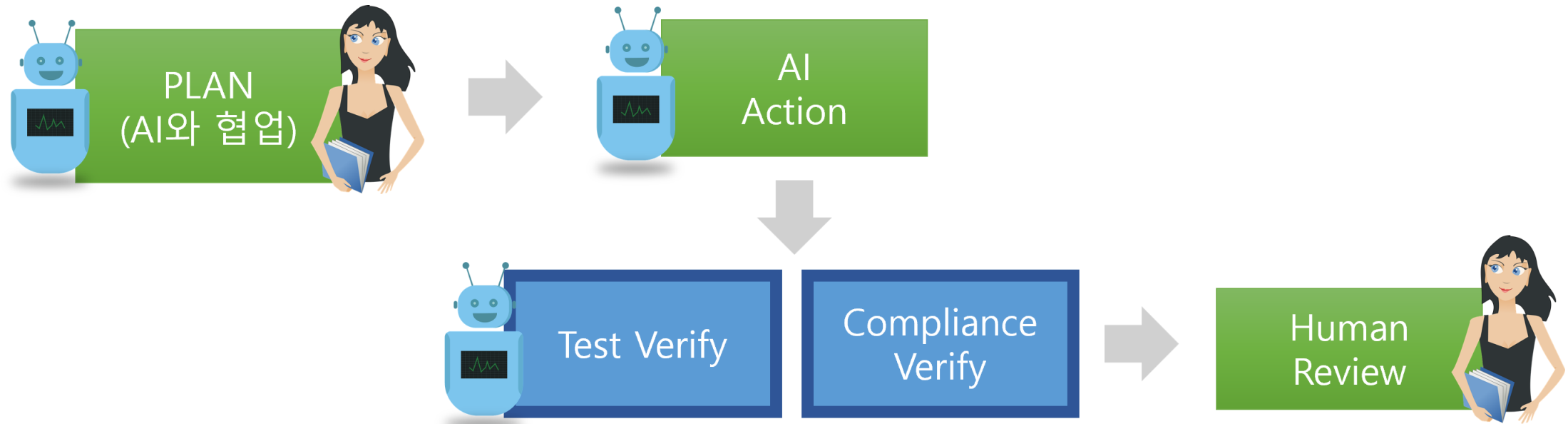
여기서 Verify는 Unit Test / E2E Test만으로는 부족하다.



Compliance Verify 도입하기

먼저 지시사항 불일치 문제를 해결하기 위해서는

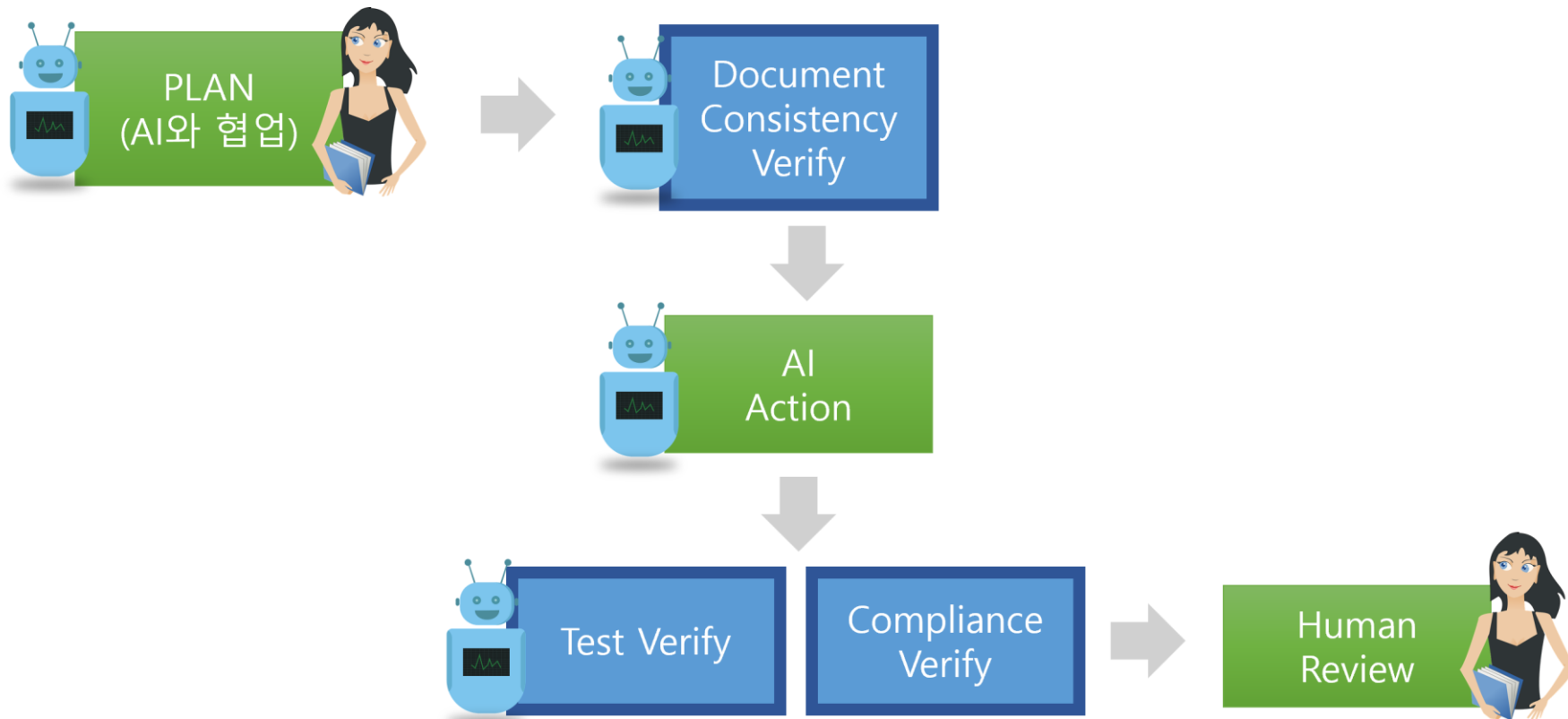
AI가 만든 코드가 PLAN 지시사항을 모두 만족하는지 검사하는 Compliance Verify (요구사항 검증)를 수행한다.



Document Consistency Verify

AI가 PLAN에 맞춰 지시를 수행하기 전에, 문서(PRD, Requirement, PLAN)들이 불일치, 충돌, 또는 모호한 경우가 있을 수 있다.

이 경우 AI는 우선순위가 높아 보이는 지시를 선택하거나, 확률 기반으로 코드를 생성하게 된다. 코드를 생성하기 전에, 지시 사항에 충돌이 있는지 사전 정합성(Consistency) 검증이 필요하다.



Chapter 05

AI 작업의 보고 체계 변경

SW 품질을 위한 Agentic Engineering

챕터의 목적

- AI가 생성한 작업물을 사람이 검토하는 과정은 Agentic Engineering의 핵심이다.
- 이를 위해 AI는 사람이 리뷰하기 쉽도록 명확한 보고 체계와 구조화된 결과물을 제공 필요
- 단순히 결과를 생성하는 것을 넘어,
검토 가능한 형태로 정리하고 설명하는 것 까지가 AI의 역할이다.

리뷰를 시작하기 위한 보고 체계를 고민하기에 앞서,
먼저 리뷰이가 리뷰어에게 검토를 받기 위해 어떤 내용을 준비해야 하는지 살펴보자.

AI 시대 이전, 사람 중심의 코드 리뷰 방식

[리뷰이가 준비할 것]

- PR 설명에 개발 의도, 변경 내용, 테스트 이력을 명확히 작성한다.
- 무엇을 왜 변경했는지, 어떤 방식으로 검증했는지 전달한다.

[리뷰어가 살펴볼 것]

- 변경된 코드와 영향 범위를 확인한다.
- 변경 내용의 적절성에 대해 리뷰이와 토론하며 개선 방향을 도출한다.

AI가 인간의 리뷰를 받기 위해 준비해야 할 것 – 작업 보고서 작성

AI 시대가 오면서 기존과 상황이 바뀌었다.

이전에 사람이 PR 템플릿 기반의 문서를 만들어서 리뷰할 때 사용한 것처럼
AI가 리뷰 받기 전, 구조화된 보고서를 준비해야 한다.

[AI가 작성해야 할 문서 내용]

- 요청 받은 작업 내용과 요구사항 정리
- 요구사항을 통해 파악한 의도 및 해석
- 변경한 내용 요약
- 핵심 주요 코드 요약
- 요청사항 충족 여부 체크리스트 생성 및 확인
- 변경된 파일 목록 / 파일별 변경 라인 수 정리

[인간(리뷰어)가 살펴볼 내용]

- AI가 작성한 보고서를 먼저 확인
- 이후 실제 변경 코드 및 영향 범위를 검토
- 필요 시 AI와 토론하며 개선 방향 도출

Chapter 06

AI 시대에 맞는 PR 전략 - 감리

SW 품질을 위한 Agentic Engineering

AI 시대 이전의 PR

AI 시대 이전에는 Pull Request(PR)를 생성하면 사람이 코드 리뷰를 수행했고, 리뷰어의 승인을 받아야만 코드를 Merge할 수 있었다.

AI 시대 이후 PR에서 나타나는 문제점 1

AI를 활용한 코드 생성으로 생산성이 급격히 증가하면서,
사람이 모든 변경 사항을 충분히 리뷰하기 어려운 상황이 발생하고 있다.

심지어 코드 작성자가 충분히 검토하지 않은 상태로 PR을 올리고,
그 부담이 그대로 리뷰어에게 전가되는 경우도 나타난다.

AI 시대 이후 PR에서 나타나는 문제점 2

AI 시대에 들어서면서 코드 작성 뿐만 아니라
코드 리뷰에도 AI 도구가 적극적으로 활용되기 시작했다.

그 결과, 기존의 PR(Pull Request) 프로세스에 대해 다음과 같은 의문이 제기되었다.

코드 리뷰를 사람이 반드시 해야 하는가?
AI가 대신할 수 있는 것 아닌가? (PR 봇)

AI가 사람보다 더 빠르고 정확하게 리뷰할 수 있다면,
인간의 PR 코드리뷰어의 역할은 무엇인가?

PR 역할의 변화

AI 도구를 활용하는 것은 자연스러운 흐름이지만, 이제 PR에서는 단순히 코드 자체를 리뷰하는 것을 넘어 다음과 같은 요소를 함께 검증해야 한다.

- 개발자가 AI가 생성한 코드에 대해 충분히 검토했는지
- 정해진 프로세스를 준수하며 책임 있게 작업했는지

즉, PR의 역할은 코드 결과물 뿐만 아니라, 코드가 만들어지는 과정과 책임까지 검증하는 방향으로 확장되고 있다.

PR 코드 리뷰 방법의 기본 방향성

AI 시대에는 코드 생산성이 크게 향상되면서, 더 이상 GitHub 상에서 코드만을 하나씩 읽으며 리뷰하는 방식에는 한계가 있다.

따라서 PR 리뷰는 코드 자체에만 집중하기보다, 실제로 동작하는 소프트웨어를 기반으로 검증하는 방식으로 전환되어야 한다.

- 실행되는 소프트웨어를 직접 확인하며 기능이 의도대로 동작하는지 검증한다.
- 관련 문서(설계, 계획, 보고서 등)가 충분히 작성되었는지, 내용이 타당한지 함께 검토한다.
- 사람이 Agentic Engineering을 잘 적용했는지 확인한다.
- PR 코드리뷰어는 AI시대에 맞는 새로운 기준을 가진 SW품질의 “감리” 역할을 해야 한다.

이제부터 AI시대의 PR 코드리뷰어는 더 구체적으로 어떤 활동을 해야 할지 살펴보기 전, 먼저 건설회사의 “감리”가 어떤 역할을 하는지 살펴보자.

건설회사의 감리는?

공사가 설계도대로 이뤄졌는지 감독하는 사람

- 공사 진행중인 현장을 직접 눈으로 확인한다.
- 설계도면 / 시방서(공사 지침서) 체크
- 문서대로 공사가 잘 진행되고 있는지 체크
- 현장에 반입된 각종 자재가 승인되고 적합한 자재만 쓰였는지?
- 업무 수칙을 잘 지키면서 공사를 이뤘는지?

건물을 잘 짓고 있는지 제3자의 눈으로 감시하여, 부실공사를 막는 핵심 인력이다.
잘못된 사항이 있다면 즉시 공사를 멈출 강한 권리를 가지고 있다.

SW로 비유하자면? (1)

업무 수칙을 잘 지키면서 공사를 진행했는가?

(담당자가 감독 없이 AI에게 모든 작업을 맡기는 방식으로 업무를 진행했는지 검사)

- GitHub에서 Commit 이력 확인
 - Commit 단위 및 순서가 Agentic Engineering 업무 프로세스에 부합하는지 확인
 - 사람이 리뷰를 제대로 했다는 Commit 도장이 제대로 찍혀 있는지 검사

공사 진행중인 현장을 직접 눈으로 확인한다.

- 개발했던 환경 그대로 로컬에 가져온다. AI와 토론할 수 있는 준비가 된다. (건설반장님으로 비유)
- AI와 토론을 통해 변경사항에 대해 완전히 이해를 한다. 가능하다면 핵심코드도 살펴본다.

설계도면 / 시방서(공사 지침서) 체크

- 목적에 맞는 Library, 자료구조, 알고리즘이 적절한지 검사
- PR Description에 브랜치의 목적, 변경내용, 체크리스트 등이 잘 되어있는지 검사
- CLAUDE.md, PRD, PLAN 등 문서의 변경사항을 확인

SW로 비유하자면? (2)

문서대로 잘 만들어 졌는지?

- AI의 도움을 받아 아래 내용을 수행한다.
 - 문서와 소스코드가 일치하는지 검사
 - PR 문서에 적은 내용과 실제 코드와 일치하는 지 검사

현장에 반입된 각종 자재가 승인되고 적합한 자재만 쓰였는지?

- AI와 토론을 하여 아래와 같은 것을 확인한다.
 - 자재는 Library, 자료구조, 알고리즘으로 비유할 수 있음
 - Library를 안 쓰고 직접 구현하거나, 과하게 자료구조 알고리즘, Library를 사용했는지 검사

PR 코드리뷰어 무게가 더 커졌다.

AI 시대가 도래하면서 검수, 검토, 감리의 책임은 과거보다 훨씬 더 무거워졌다.

- 이제 AI는 개발을 빠르게 수행하는 역할을 맡고, 사람은 그 결과를 검증하는 역할에 집중하게 된다.
- 즉, 사람의 역할은 검수와 검토 중심으로 재편되고 있다.
- AI가 구현을 담당하게 되면서 사람의 역할 범위는 오히려 좁아졌다.

하지만 그 대신, 잘못된 검수가 품질 문제로 직결되고, 잘못된 승인은 **책임**으로 이어진다.

따라서 코드리뷰어의 책임감에 대해, 모든 구성원이 명확히 이해하고 납득해야 한다.

리뷰이가 PR에 올려야 하는 것

담당자가 AI와 협업하여 개발하는 동안, AI가 만든 Report가 쌓이게 된다.
이 Report들의 통합본을 AI를 이용하여 정리하고, 글을 정돈한다.
이렇게 만들어진 정리본이 곧 PR 문서가 된다.

PLAN 문서는 Repository에 포함시키지 않는다

기능을 추가할 때 작성하는 PLAN 문서를 Git Repository에 포함시킬 것인가? —> NO 이다.

- PLAN 문서는 특정 작업을 수행하기 위한 임시 산출물이며, 최종 제품의 일부가 되면 다른 작업 시 LLM의 노이즈가 될 수 있다.
- 따라서 Repository에 포함시키는 것은 적절하지 않다.

AI 작업 보고서도 동일하다.

AI가 생성한 작업 보고서 역시 해당 작업을 수행하기 위한 과정 중심 산출물이다.

그 밖에 실행 과정, 시도 및 실패 기록, 중간 판단 흐름은 최종 소프트웨어와 직접적인 관련이 없기 때문에 직접 포함되면 역시 노이즈가 된다.

대신 이렇게 만들어진 문서는 아래와 같은 이유로 인해 버전관리를 통하여 점진적 공개를 해야 한다.

- AI와 협업한 과정에 대한 History 남기기
- 의사결정 근거에 대한 추적 가능성 확보
- 리뷰어의 이해를 돕는 보조 자료 제공

감사합니다.